



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	PUTRI REDA MAULIDIA
NPM	2428240086
Kelas	SI5B
Topik Tugas 1	Topik 21 : Kebencanaan (Laporan Bencana)
Entitas utama	disaster-reports (Laporan Bencana)
Tautan repositori	https://github.com/putrirdm/si5b_kuis1_putriredamaulidia
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	8 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Topik 21 : Kebencanaan (Laporan Bencana)
Entitas utama (tunggal)	disaster-reports
Alamat dasar	/disaster-reports
Field data	id, jenisBencana, lokasi, tanggal, tingkatKeparahan, jumlahPengungsi
Field wajib saat POST	jenisBencana, lokasi, tanggal, tingkatKeparahan
Query string untuk filter	jenisBencana : /disaster-reports?jenisBencana=banjir
Data awal	<pre>{ id: 1, jenisBencana: "banjir", lokasi: "Kec. Seberang Ulu I", tanggal: "2026-09-20", tingkatKeparahan: "sedang", jumlahPengungsi: 120 }, { id: 2, jenisBencana: "kebakaran", lokasi: "Kec. Ilir Timur I", tanggal: "2026-09-21", tingkatKeparahan: "berat", jumlahPengungsi: 80 }, { id: 3, jenisBencana: "longsor", lokasi: "Kec. Sukarami", tanggal: "2026-09-22", tingkatKeparahan: "ringan", jumlahPengungsi: 30 }</pre>
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[disaster-reports]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[disaster-reports]/:id	Tidak	200	Ambil satu data
3	POST	/[disaster-reports]	Ya	201	Tambah data
4	PUT	/[disaster-reports]/:id	Ya	200	Ubah data
5	DELETE	/[disaster-reports]/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

```

PS D:\PAW2\sisb_kuis1_putiredamaulidia> node -v
v24.21.0
PS D:\PAW2\sisb_kuis1_putiredamaulidia> npm init -y
Wrote to D:\PAW2\sisb_kuis1_putiredamaulidia\package.json:

{
  "name": "sisb_kuis1_putiredamaulidia",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/putiridm/sisb_kuis1_putiredamaulidia.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/putiridm/sisb_kuis1_putiredamaulidia/issues"
  },
  "homepage": "https://github.com/putiridm/sisb_kuis1_putiredamaulidia#readme"
}

PS D:\PAW2\sisb_kuis1_putiredamaulidia> npm install express cors dotenv
added 71 packages, and audited 72 packages in 5s

30 packages are looking for funding
run `npm fund` for details

found 0 vulnerabilities
PS D:\PAW2\sisb_kuis1_putiredamaulidia>
  
```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Jelaskan fungsi *express*, *cors*, dan *dotenv*, serta mengapa versi *Node.js* perlu memenuhi syarat *Express 5*.

Express saya gunakan untuk membuat server dan mengatur endpoint yang ada pada API, seperti GET, POST, PUT, dan DELETE.

CORS berfungsi supaya API bisa menerima request dari aplikasi atau sumber yang berbeda.

Dotenv digunakan untuk membaca isi file `.env`, misalnya untuk menyimpan PORT dan API Key agar tidak ditulis langsung di dalam program.

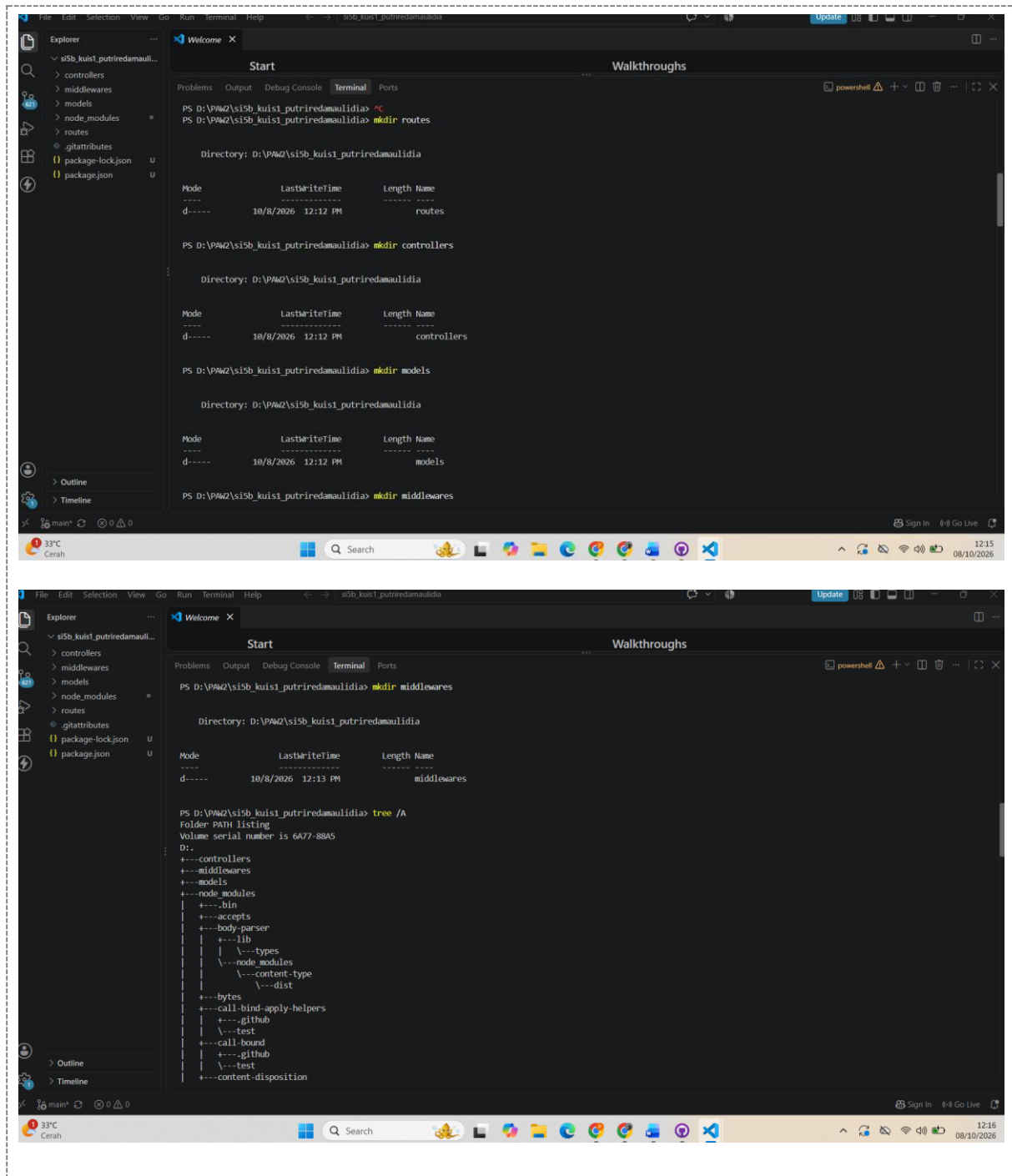
Node.js perlu sesuai dengan syarat Express 5 karena Express 5 memiliki ketentuan versi Node.js tertentu. Kalau versinya tidak sesuai, aplikasi bisa mengalami error atau tidak berjalan sebagaimana mestinya.

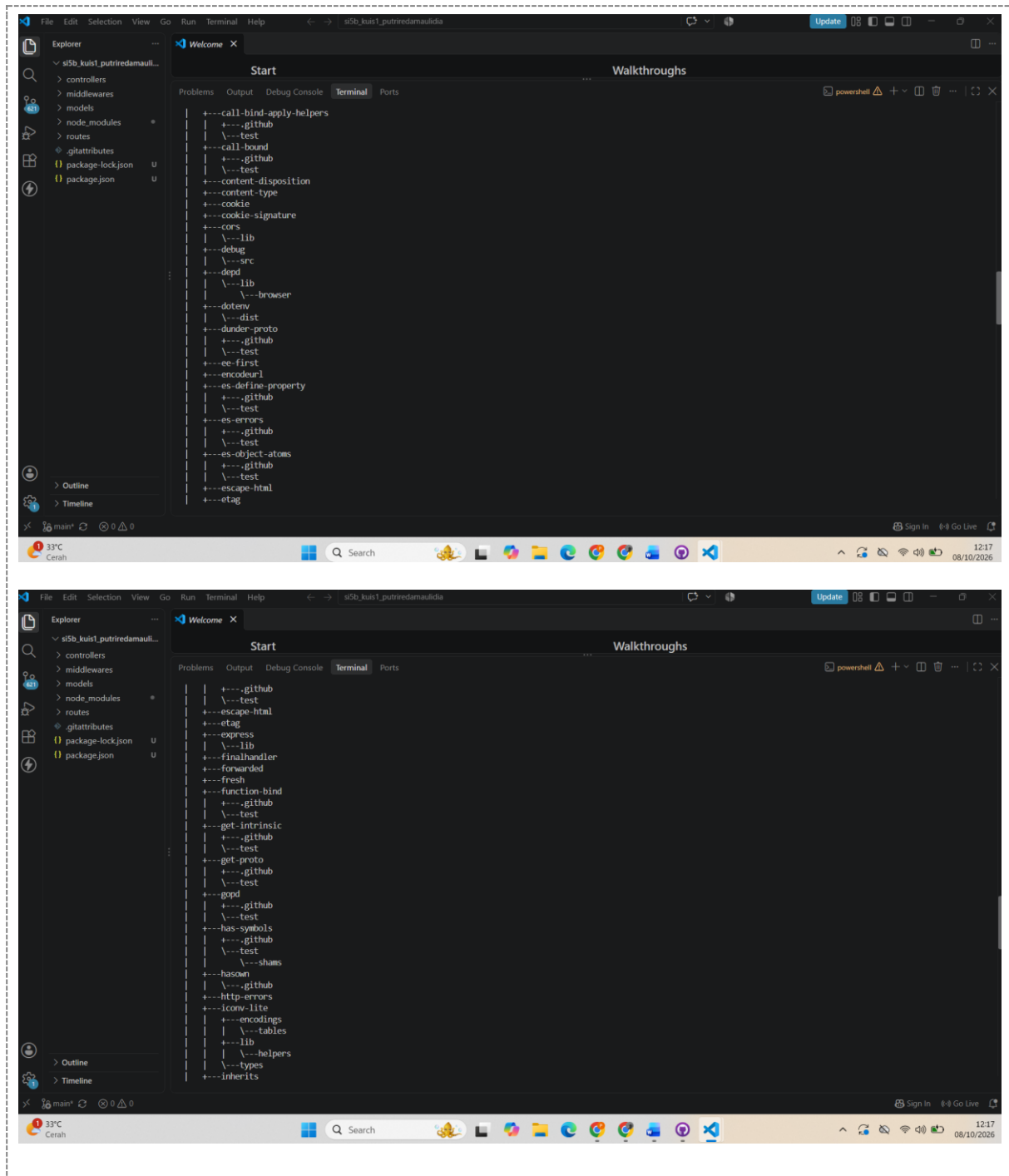
Bab 2 Struktur Folder dan Ringkasan Kode

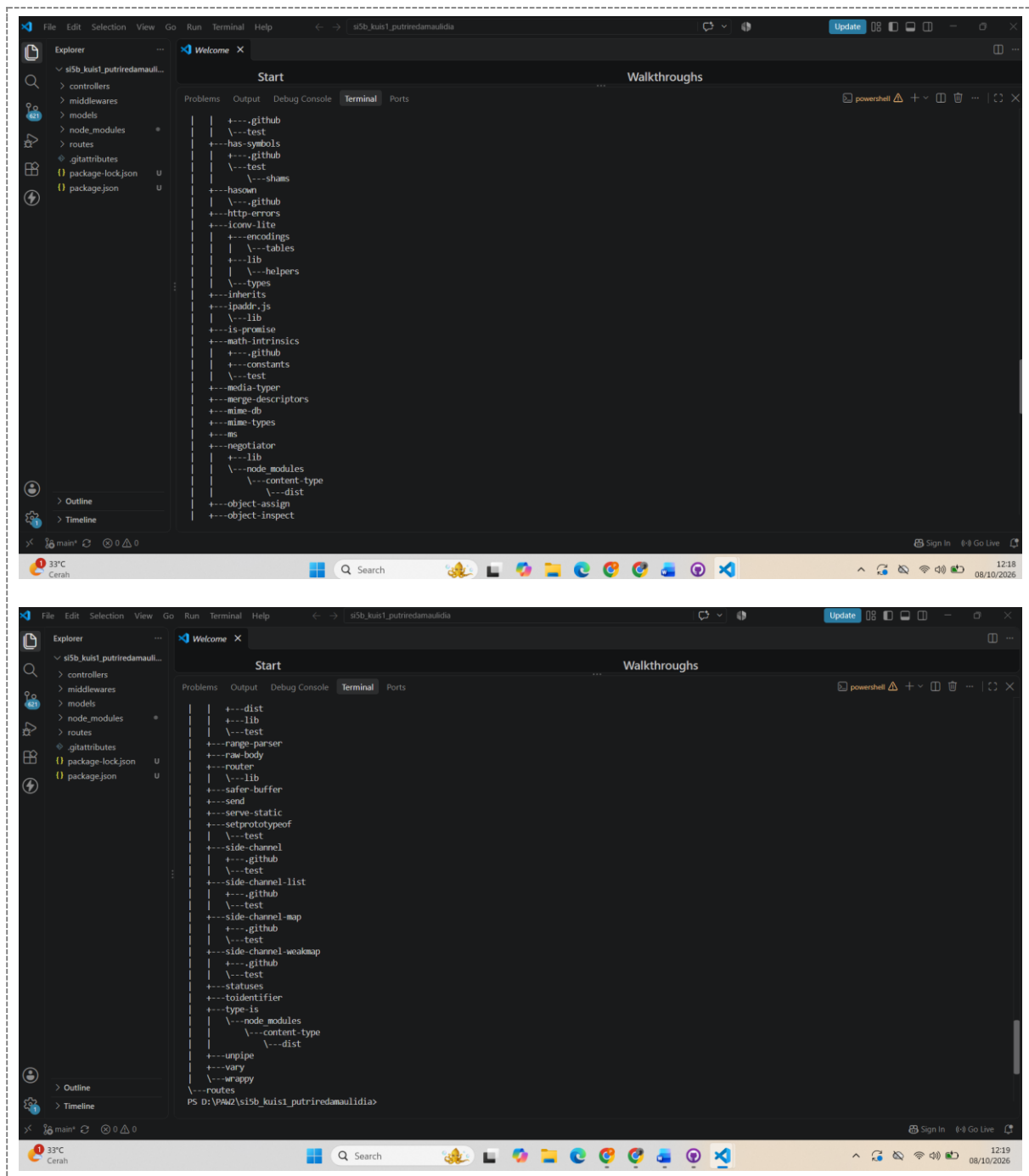
Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.







Gambar 2. Struktur folder project

Penjelasan:

Jelaskan tugas setiap folder dalam satu kalimat.

Folder **routes** digunakan untuk mengatur alamat API, **controllers** digunakan untuk mengatur proses request, **models** digunakan untuk mengelola data bencana, sedangkan **middlewares** digunakan untuk melakukan pengecekan sebelum request diproses.

2.2 Ringkasan tugas setiap berkas

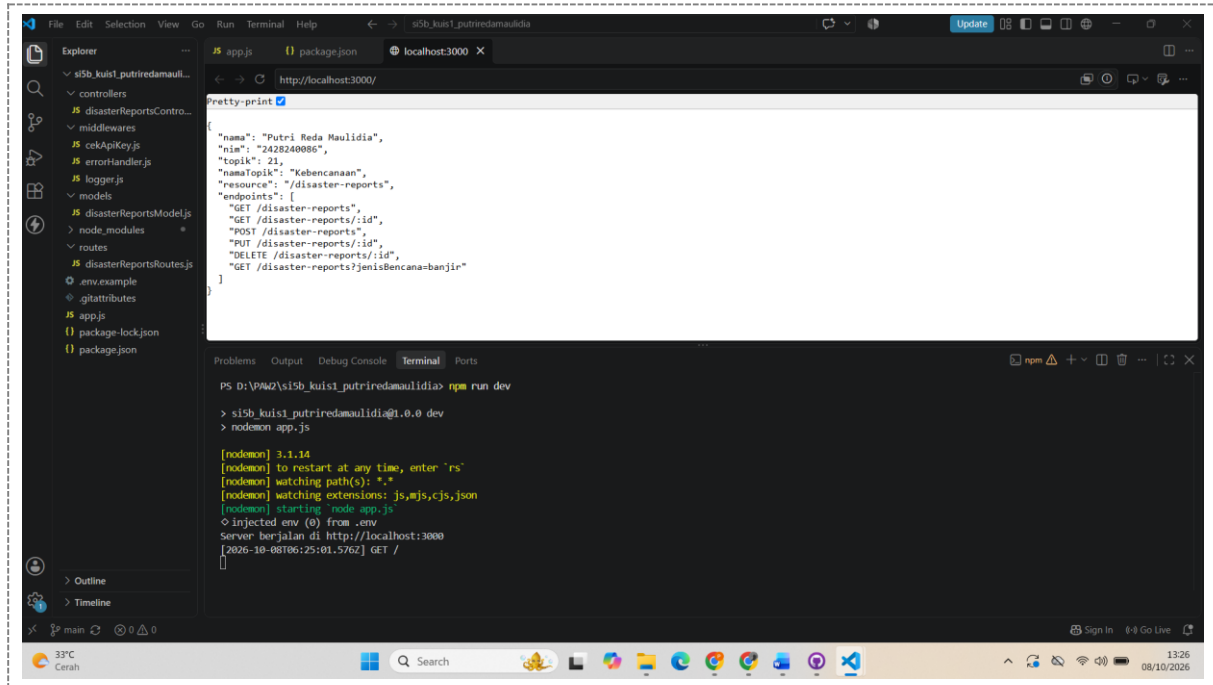
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/[sumber-daya]Model.js	Berisi data laporan bencana dan digunakan untuk mengatur data seperti menampilkan, menambah, mengubah, dan menghapus data.
controllers/[sumber-daya]Controller.js	Berisi proses untuk mengatur permintaan dari pengguna, termasuk mengecek data sebelum diproses dan memberikan hasilnya.
routes/[sumber-daya]Routes.js	Berisi daftar alamat atau endpoint yang digunakan untuk mengakses data laporan bencana.
middlewares/logger.js	Digunakan untuk melihat dan mencatat request yang masuk, seperti method dan alamat endpoint yang dipanggil.
middlewares/cekApiKey.js	Digunakan untuk mengecek API Key sebelum pengguna bisa melakukan POST, PUT, atau DELETE.
middlewares/errorHandler.js	Digunakan untuk menangani kesalahan yang terjadi agar response error yang diberikan lebih teratur.
app.js	Merupakan file utama yang mengatur jalannya aplikasi, mulai dari middleware, route, sampai menjalankan server.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Jelaskan dari mana nomor port berasal dan apa yang terjadi bila port sudah dipakai aplikasi lain.

Port yang dipakai server diambil dari file `.env`, yaitu **3000**. Kalau port tersebut sedang dipakai aplikasi lain, server akan mengalami error dan tidak bisa dijalankan di port 3000.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

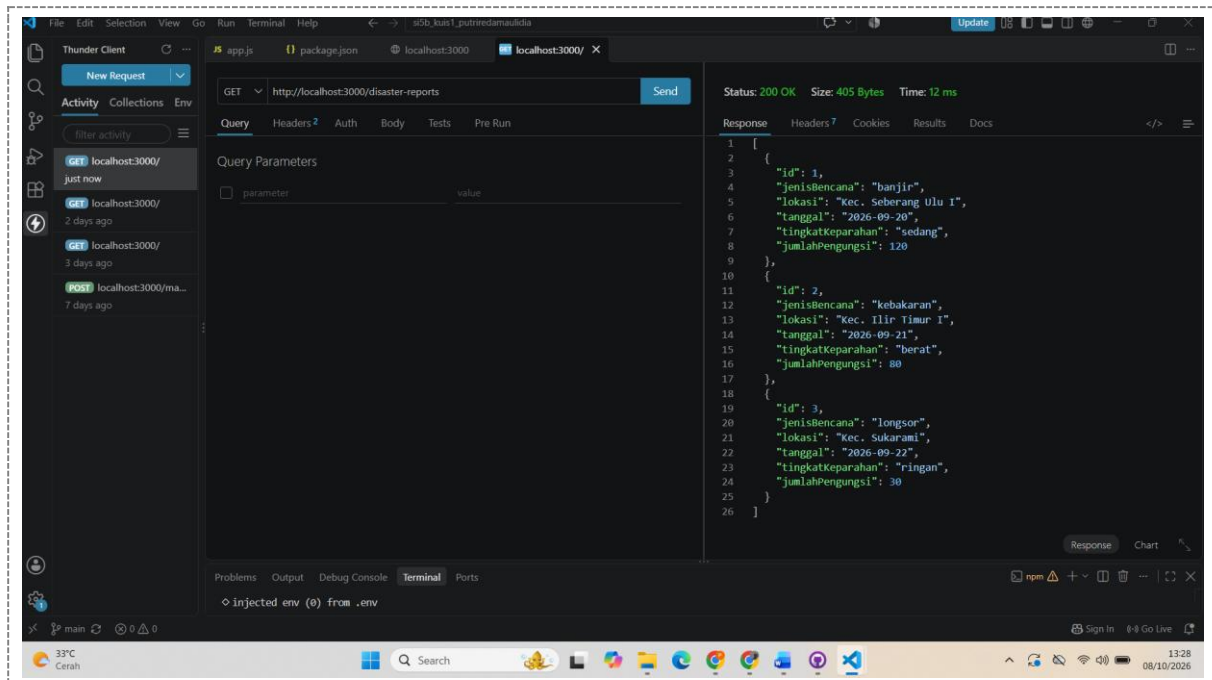
Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET `/[disaster-reports]`)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /[disaster-reports]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; [{ id: 1, jenisBencana: "banjir", lokasi: "Kec. Seberang Ulu I", tanggal: "2026-09-20", tingkatKeparahan: "sedang", jumlahPengungsi: 120 }, { id: 2, jenisBencana: "kebakaran", lokasi: "Kec. Ilir Timur I", tanggal: "2026-09-21", tingkatKeparahan: "berat", jumlahPengungsi: 80 }, { id: 3, jenisBencana: "longsor", lokasi: "Kec. Sukarami", tanggal: "2026-09-22", tingkatKeparahan: "ringan", jumlahPengungsi: 30 }]

Status hasil uji	<pre>200 OK; [{ id: 1, jenisBencana: "banjir", lokasi: "Kec. Seberang Ulu I", tanggal: "2026-09-20", tingkatKeparahan: "sedang", jumlahPengungsi: 120 }, { id: 2, jenisBencana: "kebakaran", lokasi: "Kec. Ilir Timur I", tanggal: "2026-09-21", tingkatKeparahan: "berat", jumlahPengungsi: 80 }, { id: 3, jenisBencana: "longsor", lokasi: "Kec. Sukarami", tanggal: "2026-09-22", tingkatKeparahan: "ringan", jumlahPengungsi: 30 }]</pre>
Lulus?	☒ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

Penjelasan:

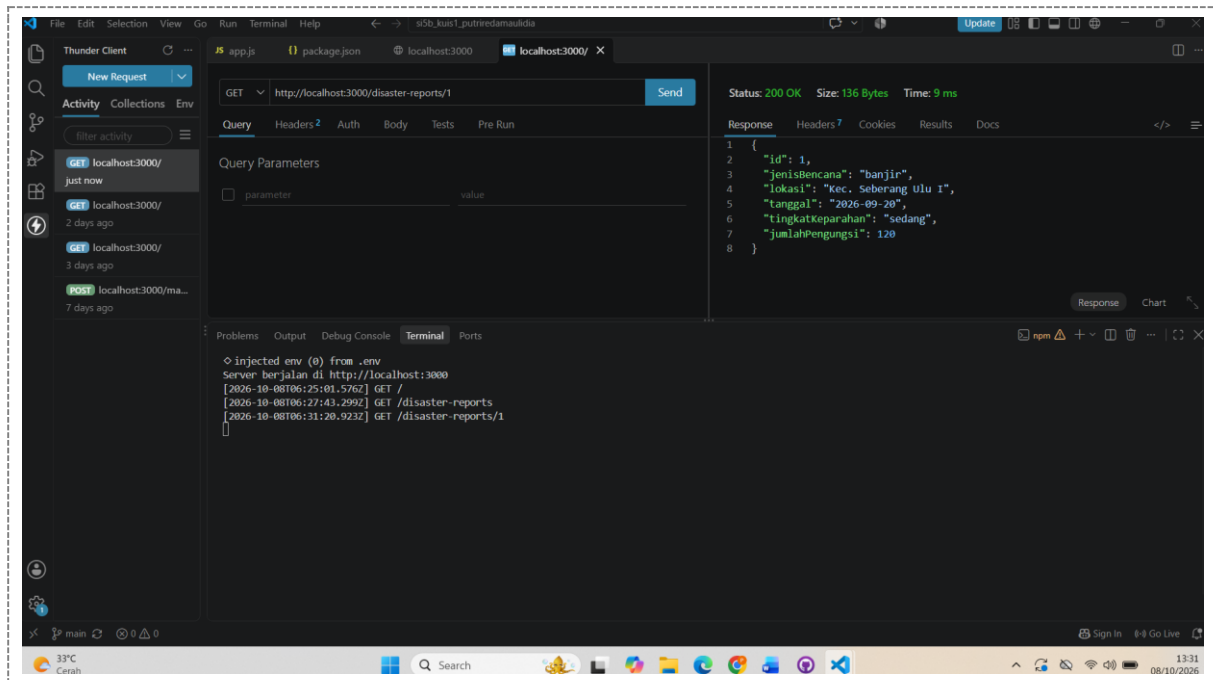
Jelaskan kode status yang muncul dan kemampuan endpoint ini. Sebutkan urutan pemanggilan: route mana, fungsi Controller mana, dan fungsi Model mana yang bekerja.

Muncul **200 OK** karena data laporan bencana berhasil ditampilkan. Endpoint ini digunakan untuk melihat semua data yang ada. Prosesnya dimulai dari **route GET /disaster-reports**, kemudian masuk ke **Controller getAll()** dan memanggil **Model getAll()** untuk mengambil data, lalu data tersebut ditampilkan sebagai hasil response.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /[disaster-reports]/[1]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; { id: 1, jenisBencana: "banjir", lokasi: "Kec. Seberang Ulu I", tanggal: "2026-09-20", tingkatKeparahan: "sedang", jumlahPengungsi: 120 }
Status hasil uji	200 OK; { id: 1, jenisBencana: "banjir", lokasi: "Kec. Seberang Ulu I", tanggal: "2026-09-20", tingkatKeparahan: "sedang", jumlahPengungsi: 120 }
Lulus?	[x] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

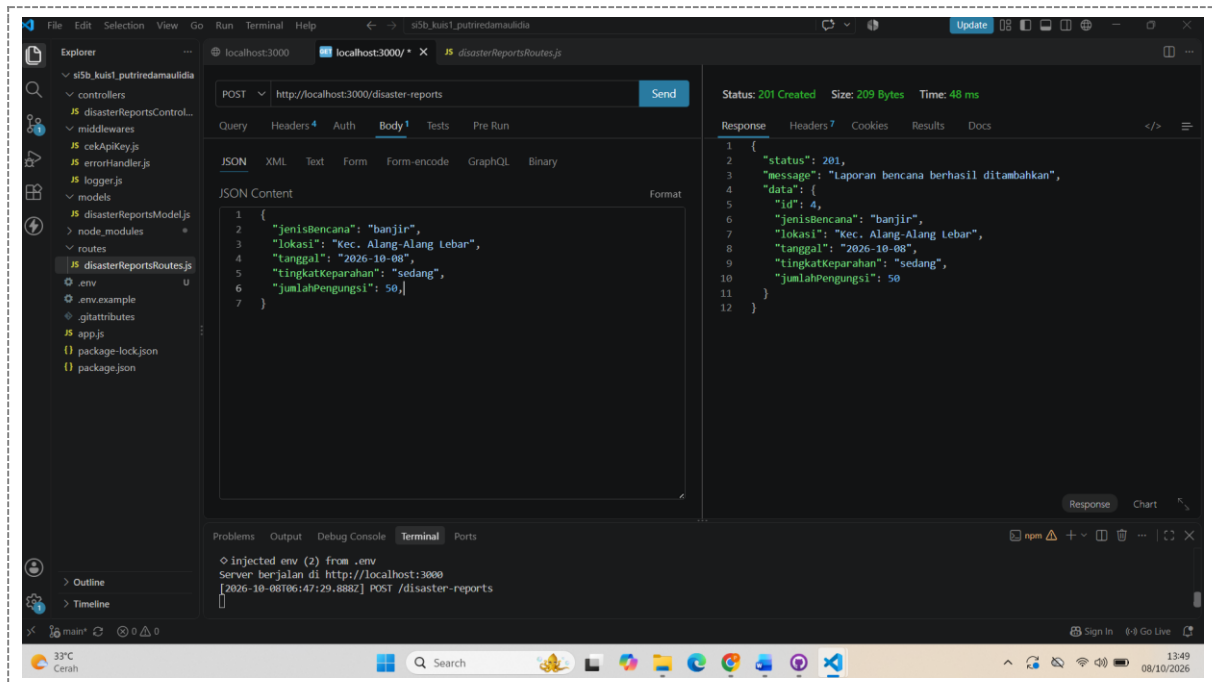
Jelaskan bagaimana ID dari alamat diambil (`req.params`) dan mengapa perlu diubah menjadi angka.

ID diambil dari URL menggunakan `req.params.id`. Karena nilai yang didapat dari URL awalnya berupa teks, ID tersebut diubah menjadi angka dengan `Number()` supaya bisa dicocokkan dengan ID yang ada di data.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /[disaster-reports]
Header	x-api-key: (kuis1_si5b_paw2) Content-Type: application/json
Body	{ "jenisBencana": "banjir", "lokasi": "Kec. Alang-Alang Lebar", "tanggal": "2026-10-08", "tingkatKeparahan": "sedang", "jumlahPengungsi": 50 }
Hasil yang diharapkan	201 Created; { "id": 4, "jenisBencana": "banjir", "lokasi": "Kec. Alang-Alang Lebar", "tanggal": "2026-10-08", "tingkatKeparahan": "sedang", "jumlahPengungsi": 50 }
Status hasil uji	201 Created dan pesan : "Laporan bencana berhasil ditambahkan"
Lulus?	[x] Ya [] Tidak



Gambar 6. Hasil POST data baru

Penjelasan:

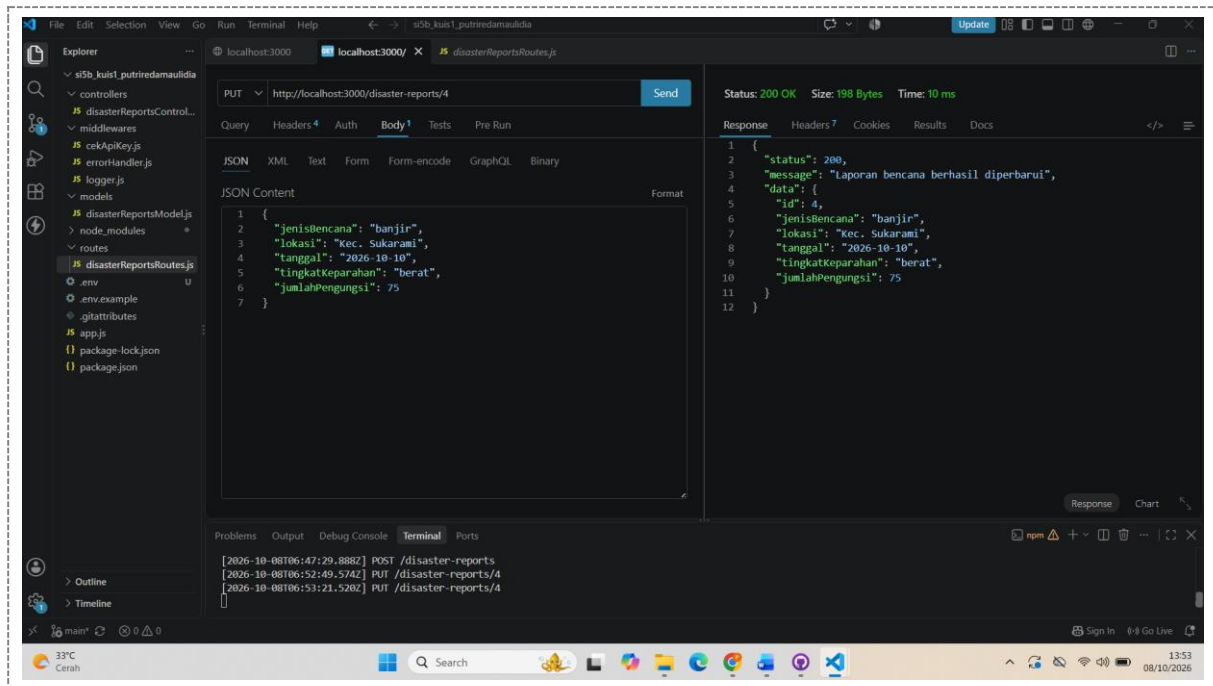
Jelaskan peran header `x-api-key` (middleware apa yang memeriksanya), peran `express.json()`, dan alasan status yang dipakai adalah 201, bukan 200.

Header `x-api-key` dipakai sebagai pengecekan akses sebelum data diproses, dan yang melakukan pengecekan adalah `cekApiKey.js`. Sementara itu, `express.json()` digunakan agar data JSON dari Body bisa diterima dan dibaca oleh server. Hasilnya menggunakan 201 Created karena pada proses ini ada data baru yang berhasil ditambahkan, sedangkan 200 OK lebih cocok untuk request yang hanya berhasil mengambil atau memproses data yang sudah ada.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (<i>request</i>)	PUT /[/disaster-reports]/[4]
Header	x-api-key: (kuis1_si5b_paw2) Content-Type: application/json
Body	{ "jenisBencana": "banjir", "lokasi": "Kec. Sukarami", "tanggal": "2026-10-10", "tingkatKeparahan": "berat", "jumlahPengungsi": 75 }
Hasil yang diharapkan	200 OK; { "id": 4, "jenisBencana": "banjir", "lokasi": "Kec. Sukarami", "tanggal": "2026-10-10", "tingkatKeparahan": "berat", "jumlahPengungsi": 75 }
Status hasil uji	200 OK dan pesan : "Laporan bencana berhasil diperbarui"
Lulus?	[x] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

Penjelasan:

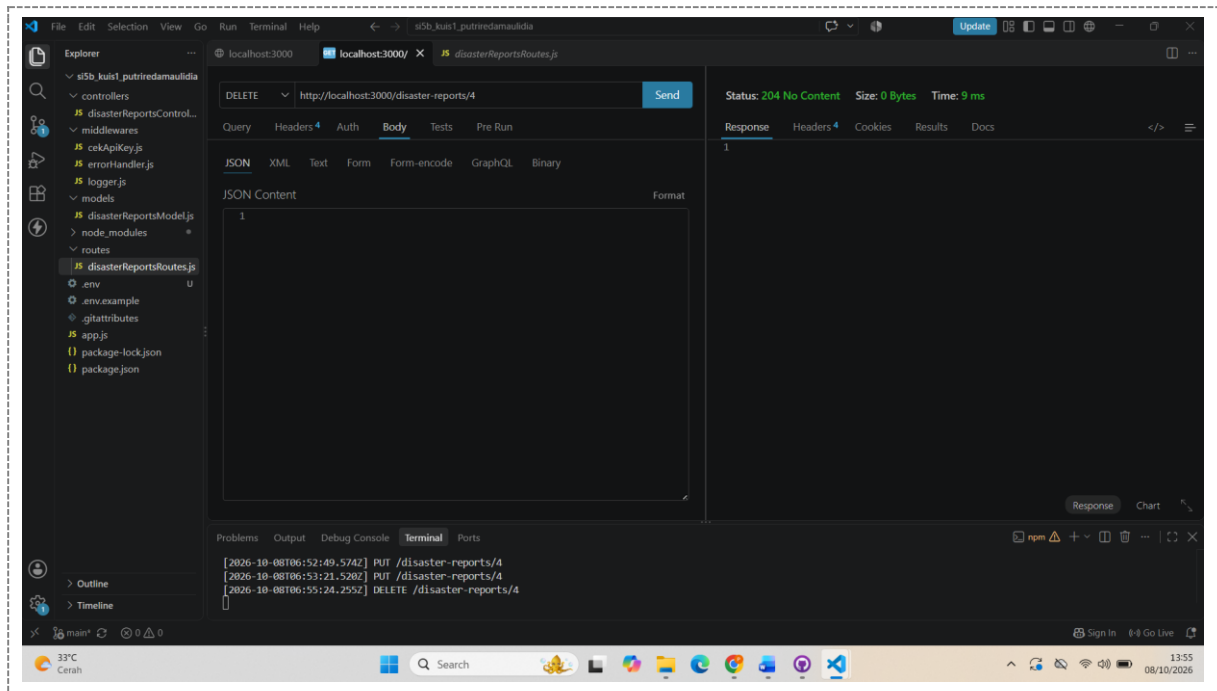
Jelaskan field apa yang berubah dan field apa yang tetap, serta layer mana yang melakukan perubahan.

Data yang berubah adalah **lokasi**, **tingkatKeparahan**, **jumlahPengungsi**, dan **tanggal**, sedangkan **jenisBencana** tetap. Perubahan data dilakukan melalui **Controller update()**, kemudian Controller meneruskannya ke **Model update()** untuk memperbarui data.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /[disaster-reports]/[4]
Header	x-api-key: (kuis1_si5b_paw2)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content
Lulus?	[x] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Jelaskan arti status 204 dan mengapa respons tidak berisi apa pun.

Status **204 No Content** berarti data berhasil dihapus. Responsnya tidak berisi apa pun karena setelah proses DELETE selesai, server tidak perlu mengirimkan data lagi kepada pengguna.

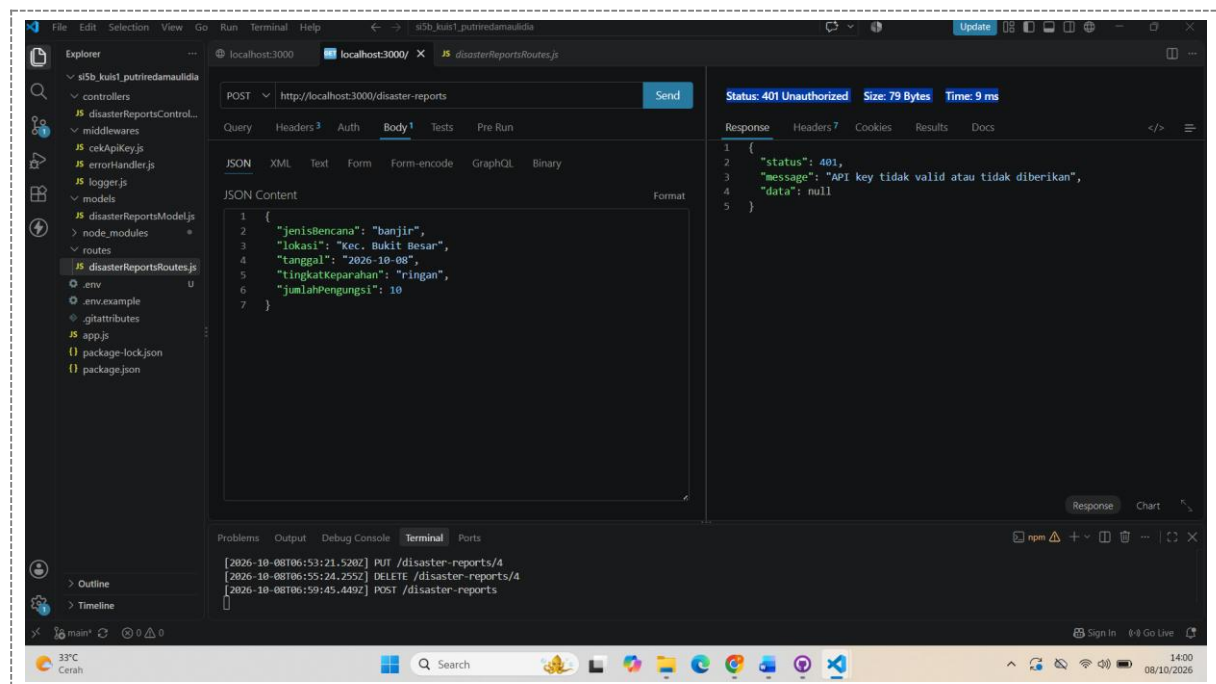
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST <code>/[disaster-reports]</code> (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ data yang valid }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized dan pesan : "API key tidak valid atau tidak diberikan"
Lulus?	[x] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

Penjelasan:

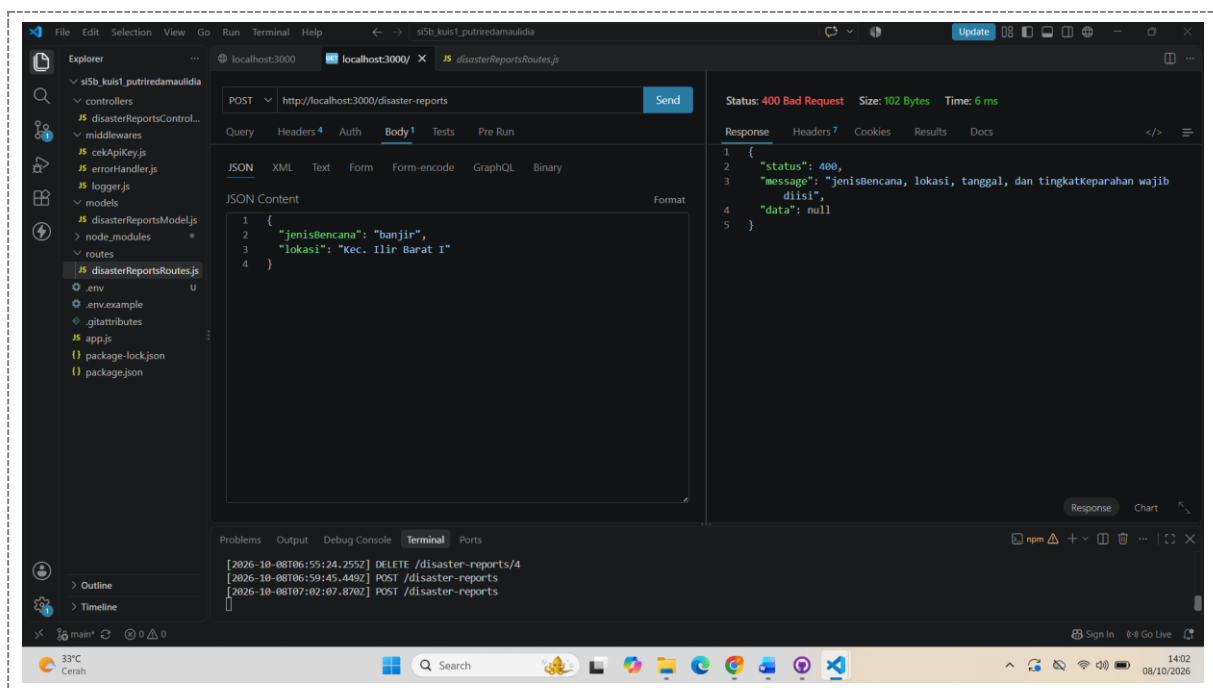
Jelaskan *middleware* mana yang menolak permintaan dan mengapa *Controller* tidak sempat dijalankan.

Request tidak bisa dilanjutkan karena **cekApiKey.js** menemukan bahwa API Key tidak ada. Akibatnya, request langsung ditolak dan **Controller create()** tidak dijalankan.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST [/disaster-reports]
Header	x-api-key: (kuis1_si5b_paw2) Content-Type: application/json
Body	{ "jenisBencana": "banjir", "lokasi": "Kec. Ilir Barat 1", }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request dan pesan : "jenisBencana, lokasi, tanggal, dan tingkatKeparahan wajib diisi"
Lulus?	[x] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

Penjelasan:

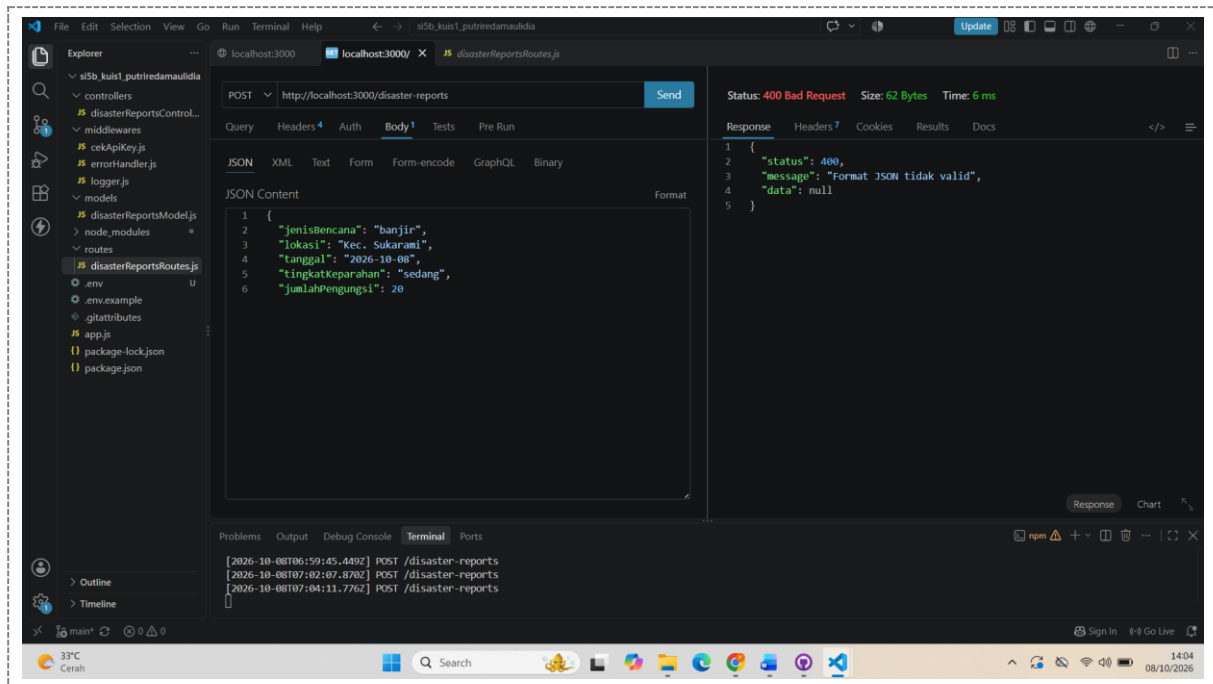
Jelaskan di layer mana validasi ini ditulis dan mengapa kode statusnya 400.

Data yang masuk diperiksa di bagian **Controller**, melalui fungsi **validate()** sebelum data disimpan. Status **400 Bad Request** muncul karena ada data yang belum diisi sehingga request tersebut dianggap tidak lengkap.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (<i>request</i>)	POST [/disaster-reports]
Header	x-api-key: (kuis1_si5b_paw2) Content-Type: application/json
Body	{ "id": 4, "jenisBencana": "banjir", "lokasi": "Kec. Sukarami", "tanggal": "2026-10-08", "tingkatKeparahan": "sedang", "jumlahPengungsi": 20
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request dan pesan : "Format JSON tidak valid"
Lulus?	[x] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

Penjelasan:

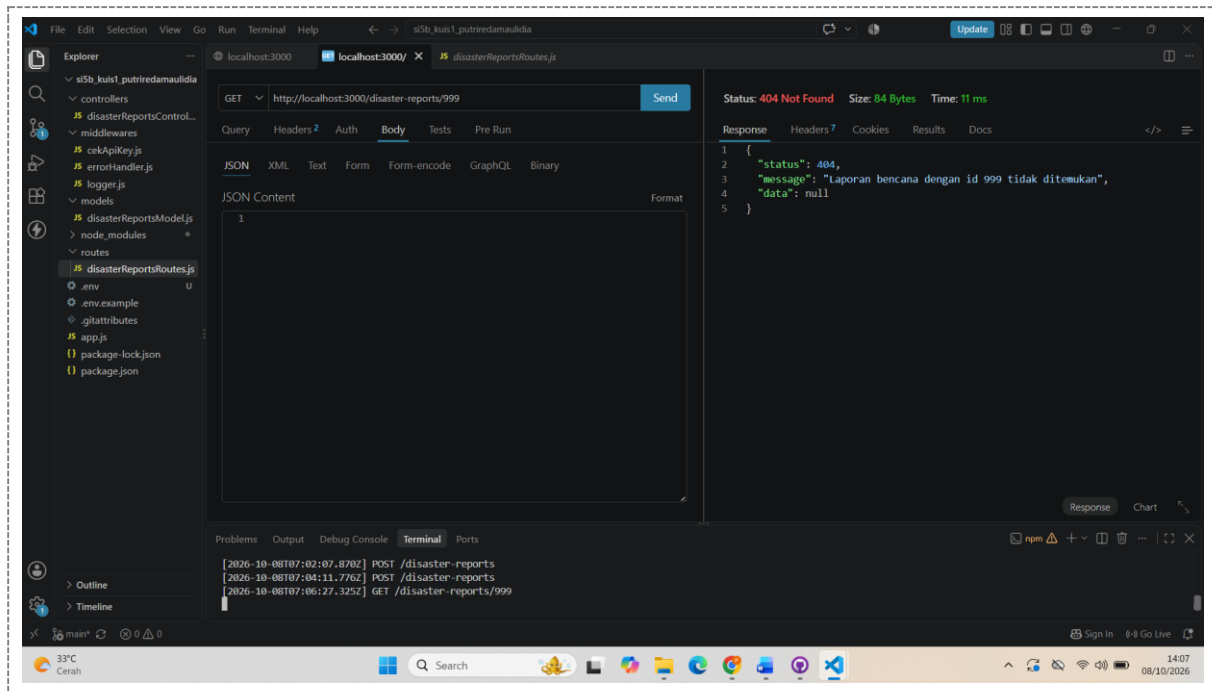
Jelaskan siapa yang mendeteksi JSON rusak ini dan bagaimana error handler mengubahnya menjadi respons yang rapi.

express.json() mengetahui bahwa format JSON yang dikirim tidak benar. Kesalahan tersebut kemudian diteruskan ke **errorHandler.js**, lalu diubah menjadi response **400 Bad Request** dengan pesan **“Format JSON tidak valid”** agar lebih mudah dipahami.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET <code>/[disaster-reports]/99</code>
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found dan pesan : “Laporan bencana dengan id 999 tidak ditemukan”.
Lulus?	[x] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Jelaskan alur kode sampai pesan 404 dibuat dan siapa yang mengirimkannya.

Request masuk melalui route **GET /disaster-reports/:id**, lalu diproses oleh **Controller getByld()**. Controller mencari ID tersebut melalui **Model getByld()**, tetapi data dengan ID 999 tidak ditemukan. Setelah itu Controller langsung mengirimkan response **404 Not Found** beserta pesan bahwa laporan bencana dengan ID tersebut tidak ditemukan.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	Lulus
T02	Menampilkan satu data berdasarkan ID	200	200	Lulus
T03	Menambah data baru	201	201	Lulus
T04	Mengubah data	200	200	Lulus
T05	Menghapus data	204	204	Lulus
N01	Menambah data tanpa API key	401	401	Lulus
N02	Menambah data dengan body tidak lengkap	400	400	Lulus
N03	Mengirim JSON yang rusak	400	400	Lulus
N04	Mengambil ID yang tidak ada	404	404	Lulus

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/putrirdm/si5b_kuis1_putrireda_maulidia
Pesan commit terakhir	Add files via upload
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

- 1) Route bertugas menentukan alamat endpoint
Contohnya: Route menggunakan `router.get("/:id", controller.getById)`
- 2) Controller mengatur proses request.
Contohnya: Controller menggunakan `model.getById(id)`
- 3) Model mengatur data laporan bencana.
Contohnya: Model menjalankan proses pencarian data berdasarkan ID.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Keuntungan:

1. Tidak perlu menulis kode penanganan error yang sama di setiap route.
2. Response error menjadi lebih rapi dan memiliki format yang sama.
3. Lebih mudah memperbaiki atau mengubah penanganan error karena cukup dilakukan pada satu file.
4. Membantu menangani berbagai jenis error yang terjadi pada API.

Bukti dari pengujian:

pada N03, saat JSON yang dikirim tidak valid, sistem memberikan 400 Bad Request dengan pesan "Format JSON tidak valid".

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	Ya
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	Ya
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	Ya
4	Identitas saya (nama atau NIM) muncul pada data uji POST	Ya
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	Ya
6	Tabel rekapitulasi sesuai dengan tangkapan layar	Ya
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggal	Ya
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	Ya

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100